

Adam AdaGrad RMSProp

AdaGrad (Adaptive Gradient Algorithm)

AdaGrad adapts the learning rate for each parameter individually based on the historical sum of squared gradients, performing larger updates for infrequent parameters and smaller updates for frequent parameters.

A student who keeps a study log:

- Subjects studied often → studies less time per session
- Subjects studied rarely → studies more time per session
- Never forgets what they've learned before

Algorithm

```
Initialize G = 0 (sum of squared gradients)
For each iteration:
    Compute gradient g = ∇θ J(θ)
    G = G + g2 (accumulate squared gradients)
    θ = θ - (η / √(G + ε)) · g
```

Mathematics

$$g_t = \nabla_{\theta} J(\theta_t)$$

$$G_t = G_{t-1} + g_t \odot g_t$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t + \epsilon}} \odot g_t$$

Example

Learning word embeddings:

- Common word “the”: G grows large → small updates
 - Rare word “quantum”: G stays small → large updates
 - **Result:** Rare features get more learning attention
-

2. RMSProp (Root Mean Square Propagation)

RMSProp improves upon AdaGrad by using an exponentially decaying average of squared gradients, preventing the learning rate from shrinking too rapidly.

Analogy

A trader with short-term memory:

- Looks at recent market trends, not ancient history
- Adapts quickly to changing conditions
- Doesn't get stuck in old patterns

Algorithm

```
Initialize  $E[g^2] = 0$ 
For each iteration:
    Compute gradient  $g = \nabla_{\theta} J(\theta)$ 
     $E[g^2] = \beta \cdot E[g^2] + (1-\beta) \cdot g^2$ 
     $\theta = \theta - (\eta / \sqrt{E[g^2] + \epsilon}) \cdot g$ 
```

Mathematics

$$g_t = \nabla_{\theta} J(\theta_t)$$

$$E[g^2]_t = \beta E[g^2]_{t-1} + (1 - \beta) g_t^2$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} g_t$$

Example

Stock price prediction:

- Recent volatility matters more than last year's
 - Learning rate adapts to current market conditions
 - **Result:** Works well with non-stationary data
-

3. Adam (Adaptive Moment Estimation)

Adam combines ideas from both Momentum and RMSProp, computing adaptive learning rates for each parameter while also using moving averages of gradients (momentum).

Analogy

A self-driving car with smart cruise control:

- Builds speed on clear roads (momentum)
- Slows down for curves and obstacles (adaptive learning)
- Smooth and efficient navigation

Algorithm

Initialize $m=0$, $v=0$, $t=0$

For each iteration:

$t = t + 1$

Compute gradient $g = \nabla_{\theta} J(\theta)$

$m = \beta_1 \cdot m + (1-\beta_1) \cdot g$ (1st moment - mean)

$v = \beta_2 \cdot v + (1-\beta_2) \cdot g^2$ (2nd moment - variance)

```
m_hat = m / (1-β1t)           (bias correction)
v_hat = v / (1-β2t)
θ = θ - η · m_hat / (√v_hat + ε)
```

Mathematics

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

Example

Training a deep neural network:

- Builds momentum in consistent gradient directions
- Adapts step size for each parameter individually
- **Result:** Fast convergence with minimal tuning

Comparative Analysis

Algorithm	Learning Rate	Momentum	Gradient History	Strengths	Weaknesses
SGD	Fixed	✗ No	✗ None	Simple, theoretical	Slow, sensitive to

Algorithm	Learning Rate	Momentum	Gradient History	Strengths	Weaknesses
				guarantees	LR
Momentum	Fixed	✓ Yes	✗ Short-term	Faster convergence, handles ravines	Still needs manual LR tuning
AdaGrad	Adaptive	✗ No	✓ All history	Great for sparse data	LR decreases to zero
RMSProp	Adaptive	✗ No	✓ Recent	Handles non-stationary objectives	No momentum component
Adam	Adaptive	✓ Yes	✓ Recent	Fast, robust, minimal tuning	Can sometimes generalize worse

Key Technical Differences

Aspect	SGD	Momentum	AdaGrad	RMSProp	Adam
LR Adaptation	Global	Global	Per-parameter	Per-parameter	Per-parameter
Gradient Memory	None	Exponential decay	Full accumulation	Exponential decay	Exponential decay
Velocity Term	No	Yes	No	No	Yes
Bias Correction	No	No	No	No	Yes

Aspect	SGD	Momentum	AdaGrad	RMSProp	Adam
Default Choice	Rarely	Sometimes	NLP tasks	RNNs	Most cases

TensorFlow Implementation

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Model architecture
model = Sequential([
    Dense(128, activation='relu', input_shape=(10,)),
    Dense(64, activation='relu'),
    Dense(1, activation='sigmoid')
])

# Optimizer configurations
optimizers = {
    'sgd': tf.keras.optimizers.SGD(learning_rate=0.01),
    'momentum': tf.keras.optimizers.SGD(learning_rate=0.01, momentum=0.9),
    'adagrad': tf.keras.optimizers.Adagrad(learning_rate=0.01),
    'rmsprop': tf.keras.optimizers.RMSprop(learning_rate=0.001, rho=0.9),
    'adam': tf.keras.optimizers.Adam(learning_rate=0.001, beta_1=0.9, beta_2=0.999)
}

# Compile with Adam (recommended default)
model.compile(
    optimizer=optimizers['adam'],
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```